

Week 1 - Day 3a: While Loops

You will be working on a series of exercises related to while loops

Lecture Recap: While Loops

A **while loop** is a control flow statement that allows us to execute code repeatedly based on a given condition.

The syntax for while loops is:

```
while <condition>:  
    <body>
```

As with **if** statements, the **<condition>** can be anything that evaluates to a boolean. The **<body>** can be complex.

The body of the loop is repeated over and over again, as long as the **<condition>** is **True**. Once the condition is false, the loop stops, and whatever comes after the body is executed next.

Exercise 1: Intro to While Loops []

1.1 Look at the code below. How often is the loop body executed? Think about it before running the code.

```
i = 0  
while i < 5:  
    print("Hello World")  
    i += 1
```

1.2 Look at the code below. How often is the loop body executed now? Think about it before running the code.

```
i = 6  
while i < 10:  
    print("Hello World")  
    i += 1
```

1.3 Look at the code below. How often is the loop body executed now? Think about it before running the code.

```
i = 0  
while i < 10:  
    print("Hello World")  
    i += 3
```

Exercise 2: What if Our Condition is a Direct Boolean? []

2.1 If we have a loop

```
while True:
    <body>
```

How often is the loop body executed?

2.2 If we have a loop

```
while False:
    <body>
```

How often is the loop body executed?

Let's test it out! Try running the code below and see what happens.

*NB: If your code does not stop running and has an `[*]` on the left, it is in an infinite loop. To get out of this loop, in the top menu, select **Kernel -> Interrupt**, or press the **black square button** in the top menu bar.*

```
i = 0
while True: # Line 2
    i += 1
print(i)
```

2.3 How often is the body (`i += 1`) executed?

2.4 What would the output be if you change Line 2 to `while False:`? First guess, and then confirm by running the code cell below. How often is the body (`i += 1`) executed now?

```
i = 0
while False: # Line 2
    i += 1
print(i)
```

Exercise 3: Fix the Code []

In each of the following code snippets, there is a mistake. Find it and fix it!

Suggestion: First run the code and look at the error message. Fix it and run it again.

3.1 The goal is to print `Hello World` five times, and then `Goodbye`. Find the mistake and fix it.

```
# TODO: Find and fix the mistake
i = 0
while i < 5
    i += 1
```

```
print("Hello World")
print("Goodbye")
```

3.2 The goal is to print Hello World five times, and then Goodbye. Find the mistake and fix it.

```
# TODO: Find and fix the mistake
i = 0
while i < 5:
    print("Hello World")
    i += 1
    print("Goodbye")
```

3.3 The goal is to print Hello World five times, and then Goodbye. Find the mistake and fix it.

```
# TODO: Find and fix the mistake
i = 0
while i < 5:
    print("Hello World")
    i += 1
    print("Goodbye")
```

3.4 The goal is to print Hello World five times, and then Goodbye. Find the mistake and fix it.

```
# TODO: Find and fix the mistake
i = 0
while i < 5:
    print("Hello World")
    i + 1
    print("Goodbye")
```

3.5 The goal is to print Hello World five times, and then Goodbye. Find the mistake and fix it.

```
# TODO: Find and fix the mistake
i = 0
while i > 5:
    i += 1
    print("Hello World")
    print("Goodbye")
```

3.6 The goal is to print Hello World five times, and then Goodbye. Find the mistake and fix it.

```
# TODO: Find and fix the mistake
i = 0
while i < 5:
    i += 1
    print("Hello World")
    print("Goodbye")
```

Exercise 4: Write Your Own While Loops []

4.1 Print the numbers between 0 to 30 (including both 0 and 30).

```
# TODO: Your code here
```

4.2 Print the **even** numbers between 0 to 30 (including both 0 and 30).

Hint: You only have to change a single symbol from the previous answer!

```
# TODO: Your code here
```

4.3 Print all powers of 2 less than 100.

```
# TODO: Your code here
```

4.4 Print all the numbers in the list `lst=[41, 2, 89, 50, 12, 13]` using a `while` loop:

Hint: Use a number `i` that goes from 0 to 5, and get the list elements like `myList[i]`

```
# TODO: Your code here
```

Lecture Recap: While Loops (cont.)

We can have more complicated logic inside the loop, like `if` statements.

The following code prints all numbers from the list which are greater than 20:

```
lst = [41, 2, 89, 50, 12, 13]
i = 0
while i < len(lst):
    if lst[i] > 20:
        print(lst[i])
    i += 1
```

Exercise 5: While Loops with an If Statement []

5.1 Print all the **even** numbers in the list `lst=[41, 2, 89, 50, 12, 13]` using a while loop.

Hint: Use an `if` statement inside your `while` loop

```
# TODO: Your code here
```

5.2 Use a while loop to print all numbers from 1 to 99 where the last digit is 3 or 7 (e.g. 23 will be printed but not 24).

Hint: Use an `if` statement inside your `while` loop, and `or` inside your condition.

```
# TODO: Your code here
```

Exercise 6: Sum Checking []

When computing $1+2+3+4+\dots$, how many numbers do you need to sum up so that the sum exceeds 1000? For example, for the sum to exceed 12 we need 5 numbers, because $1+2+3+4+5=15>12$

Hint: Use a while loop

```
# TODO: Your code here
```

Exercise 7: Fix the Code []

In each of the following code snippets, there is a mistake. Find it and fix it!

Suggestion: First run the code and look at the error message. Fix it and run it again.

7.1 The code should sum up the first 10 square numbers ($1^1+2^2+\dots+10^2$), but the loops runs infinitely. Find and fix the mistake.

```
# TODO: Find and fix the mistake
i = 1
summ = 0
while i <= 10:
    summ += i**2
print(summ)
```

7.2 The code should add (and print) the natural numbers (1, 2, 3, ...) until their sum exceeds 100 but nothing is printed. Find and fix the mistake.

```
# TODO: Find and fix the mistake
total = 0
i = 1
while total >= 100:
    total += i
    i += 1
    print(i)
```

7.3 The code should print the first number in the list `lst` that is bigger than 10. Find and fix the mistake.

```
# TODO: Find and fix the mistake
lst = [1, 3, 9, 6, -1, 13, 6, 11, 2]
number = 0
while lst[number] <= 10:
    number += 1
print(number)
```

Exercise 8: Check List Elements []

Use a while loop to keep printing numbers in a list until you find a number greater than 10. For example, given this list `lst = [1, 7, 4, 5, 15, 2, 3, 6, 9, 12]`, your program should print

```
1
7
4
5
# TODO: Your code here
```

Exercise 9: Fibonacci Sequence []

The Fibonacci sequence is defined as follows: The first two elements in the sequence $f(0)=0$ and $f(1)=1$. For every $n>1$, the n -th element in the sequence is

$$f(n)=f(n-1)+f(n-2).$$

Use a while loop to write a code that prints out all the Fibonacci numbers less than 10000.

```
# TODO: Your code
```

Exercise 10: Collatz Sequence []

The Collatz sequence of a number n is defined as follows:

- It starts with the number $a_0=n$ itself
- If the i -th number a_i is even, then the next number a_{i+1} is $\frac{a_i}{2}$
- If the i -th number a_i is odd, then the next number a_{i+1} is $3 \cdot a_i + 1$
- The sequence ends once you reach the number 1.

For example: For the number 12, the sequence is

```
12, 6, 3, 10, 5, 16, 8, 4, 2, 1
```

The Collatz conjecture says that whichever number you start with, you'll always reach 1 at some point. Define an integer variable `n` and then use a `while` loop to print out its Collatz sequence!

```
# TODO: Your code here
```

Exercise 11: Guessing Loop Output[]

You are given the while loop below.

```

x, y, loops = 0, 21, 0
while y:
    loops += 1
    if x <= y:
        if x % 2 == 1:
            x, y = y, x
        elif x == 0:
            x += (2 ** 2)
        else:
            x = x + 3
            y = y // 2
    else:
        y = 0

```

11.1 Manually go through the loop and predict what `x` and `loops` will output. Do NOT check your answer until you have predicted the output.

11.2 Try the code above to see if you are correct.

Run the code to check your predictions

```

x, y, loops = 0, 21, 0

while y:
    loops += 1
    if x <= y:
        if x % 2 == 1:
            x, y = y, x
        elif x == 0:
            x += (2 ** 2)
        else:
            x = x + 3
            y = y // 2
    else:
        y = 0

print(f"x: {x}, loops: {loops}")
x: 10, loops: 4

```

Congratulations 🎉